

L.EIC006

Tratamento Paralelo de Dados

João Canas Ferreira / António José Araújo

Abril de 2025

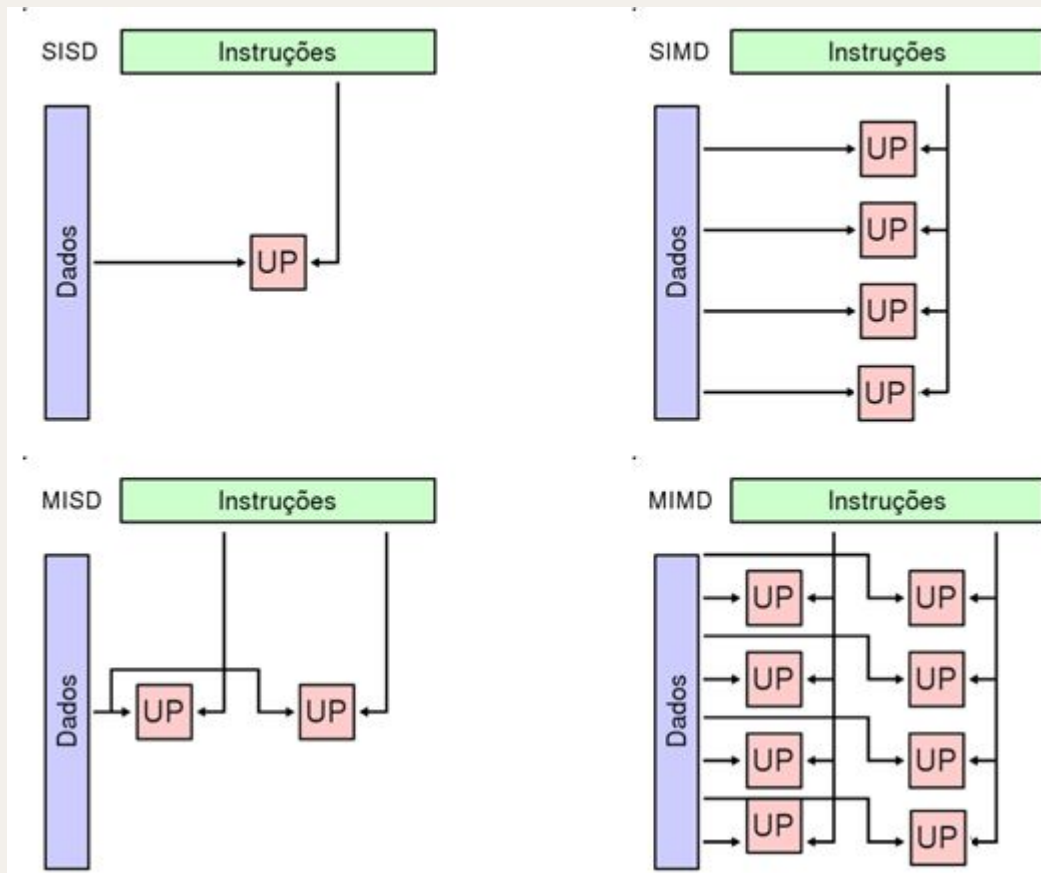


Modelo de Computação

Modelos de execução de instruções

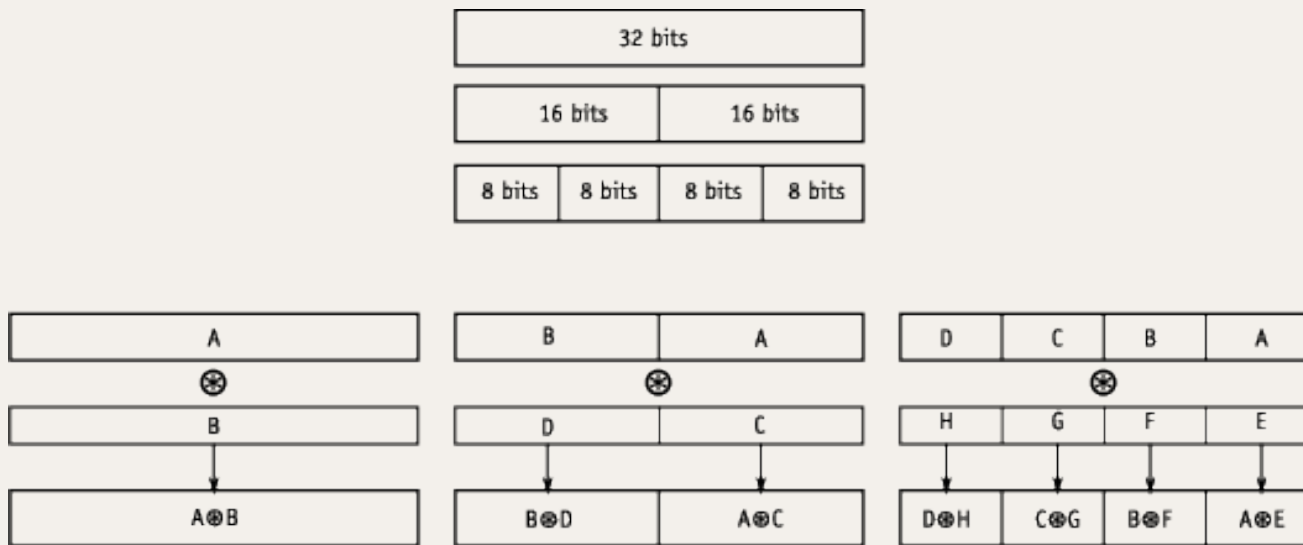
- As unidades de processamento podem ser classificadas segundo o número de instruções e de conjuntos de dados tratados em simultâneo.
- **SISD:** *Single instruction stream, single data stream*
Processador convencional: 1 instrução processa 1 conjunto (par) de dados
- **SIMD:** *Single instruction stream, multiple data streams*
Uma instrução processa vários conjuntos de dados
(por exemplo, processamento vetorial)
MISD: *Multiple instruction streams, single data stream*
Exemplo: processamento redundante (pouco usado)
- **MIMD:** *Multiple instruction streams, multiple data streams*
múltiplas instruções (diferentes) processam múltiplos conjuntos de dados
(por exemplo, um processador multinúcleo)

Representação dos modelos de execução



Instruções SIMD: modelo abstrato

- O conteúdo de um registro pode ser interpretado como uma unidade (interpretação tradicional) ou como um vetor com um número fixo (p. ex., 2 ou 4) de componentes **independentes**.
- Cada elemento do vetor pode ser combinado com o elemento correspondente de outro vetor com **uma única instrução**.



Instruções SIMD: vantagens e desvantagens

Vantagens

- Aumento de desempenho (mais dados processados por instrução)
- Aproveitamento da capacidade de integração (capacidade de integrar número elevado de ALUs e outras unidades de processamento)
- Paralelismo explícito é fácil de aproveitar (operações são naturalmente independentes)

Desvantagens

- Aplicáveis apenas em algoritmos com processamento de dados “regular” (todos os elementos do vetor são processados da mesma forma)
- Necessidade de adaptar a codificação do algoritmo
- Compiladores podem ter dificuldade em aproveitar bem este tipo de instruções: recurso a linguagem *assembly*.
- Limitação de tamanhos (registos de tamanho fixo)

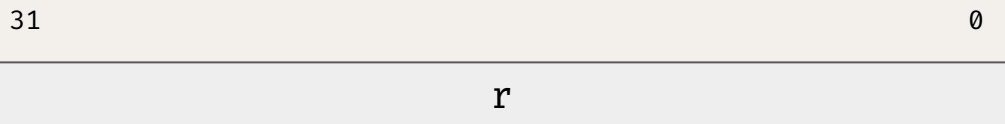
Instruções SIMD

Princípios básicos

- Extensão “P” do conjunto de instruções RISC-V
 - P: *packed SIMD instructions*
- Usam os registos de números inteiros. Para a versão de 32 bits:
 - 2 números de 16 bits
 - 4 números de 8 bits
- Tipos de instruções
 - Adição, subtração, multiplicação, deslocamento
 - Comparação
 - Empacotamento/desempacotamento
 - Outras

Notação básica

1 registo de 32 bits



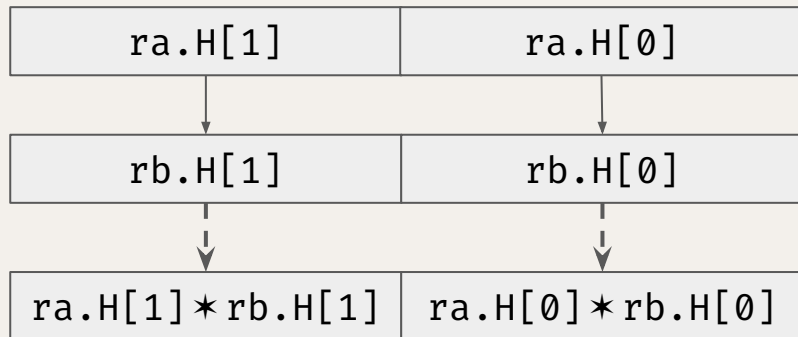
2 registos de 16 bits



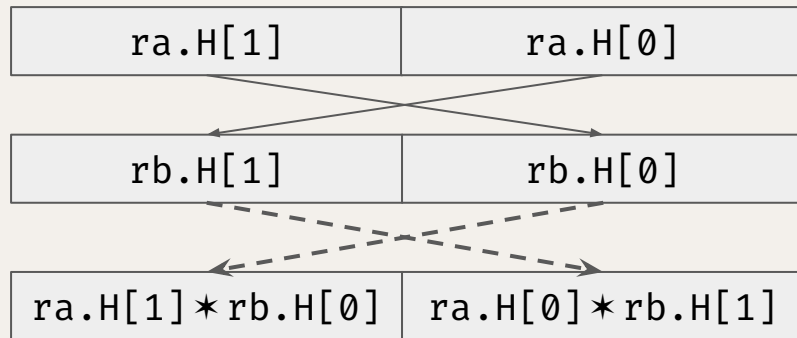
4 registos de 8 bits



Operação direta

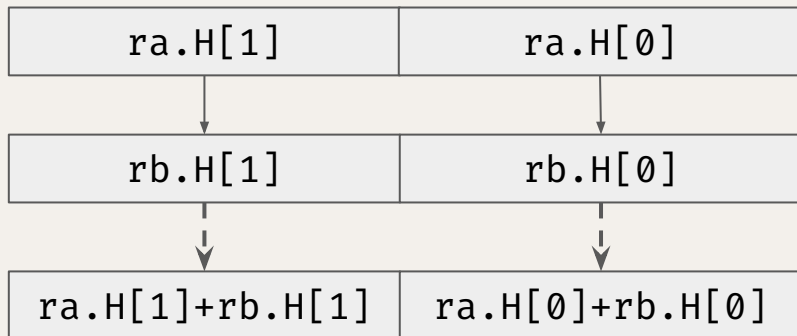


Operação cruzada

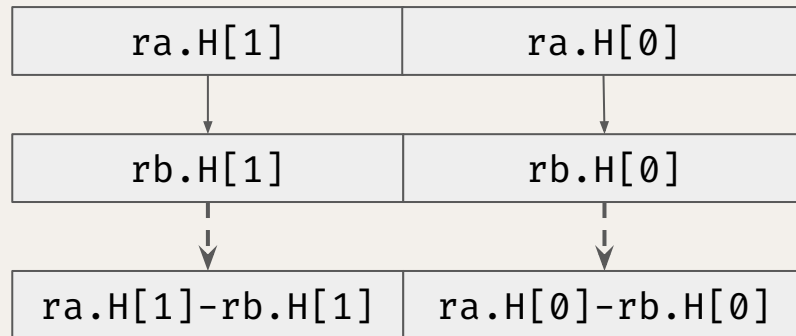


Adição e subtração de 16 bits (I)

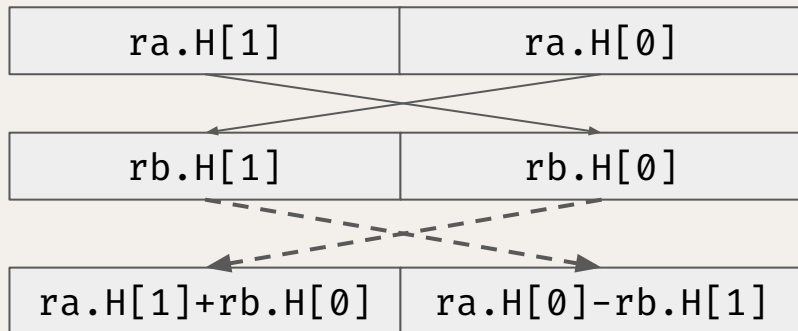
ADD16 rd,ra,rb



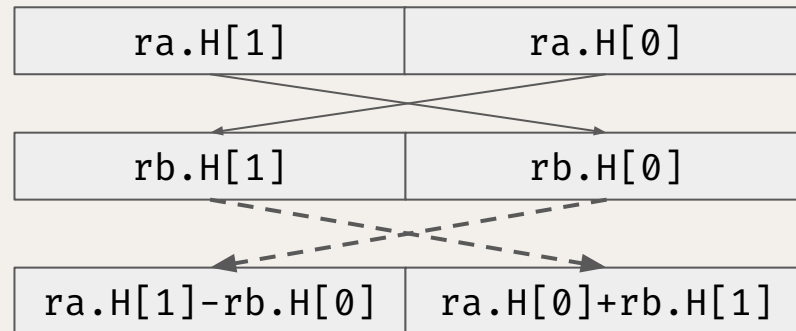
SUB16 rd,ra,rb



CRAS16 rd,ra,rb

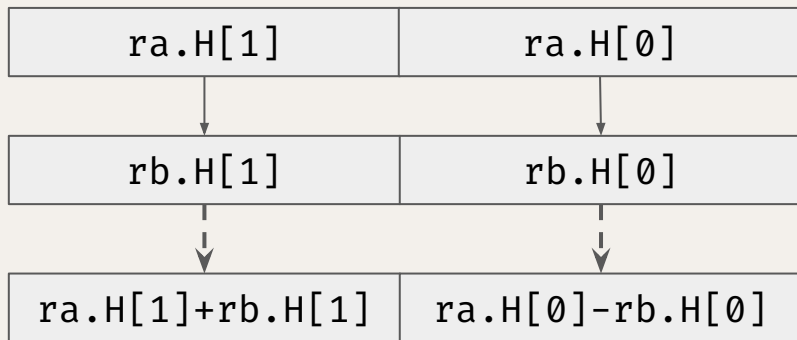


CRSA16 rd,ra,rb



Adição e subtração de 16 bits (II)

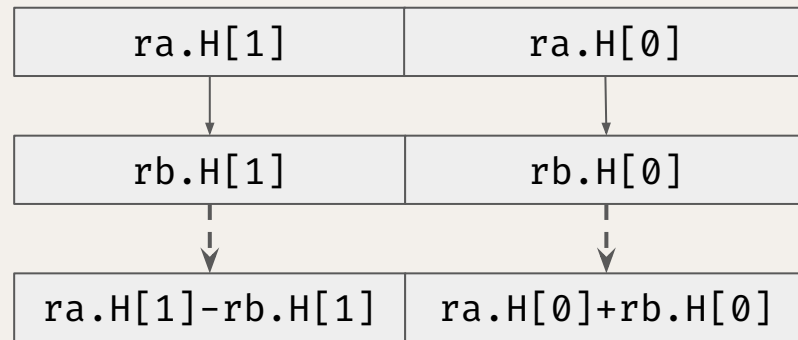
STAS16 rd,ra,rb



adição

subtração

STSA16 rd,ra,rb

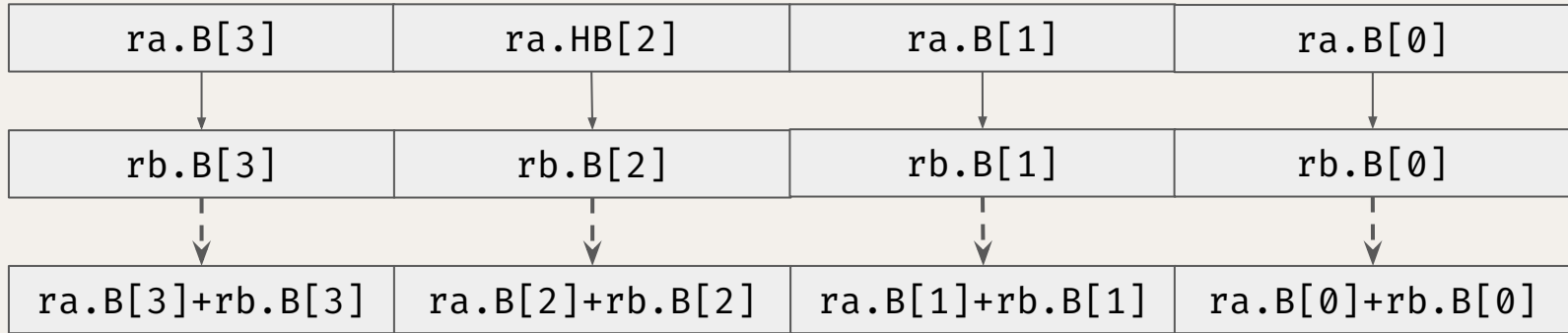


subtração

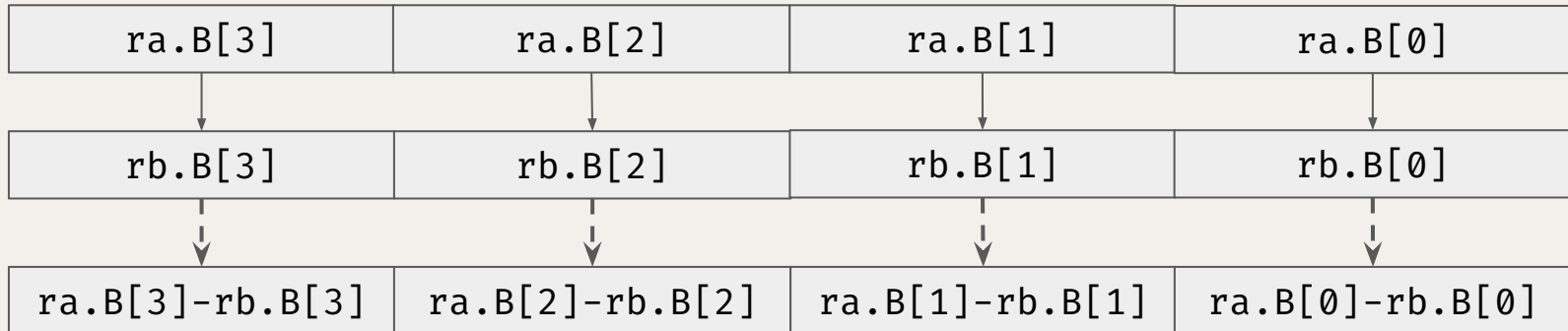
adição

Adição e subtração de 8 bits

ADD8 rd,ra,rb



SUB8 rd,ra,rb



Instruções de deslocamento

Mnemônica	Operação	Resultado
SRA16 rd,rs1,rs2	<i>Shift right arithmetic</i> (propaga sinal)	rs1.H[0]>>rs2[3:0], rs1.H[1]>>rs2[3:0]
SRAI16 rd,rs1,imm4	<i>Shift right arithmetic immediate</i> (propaga sinal)	rs1.H[0]>>imm4, rs1.H[1]>>imm4
SRL16 rd,rs1,rs2	<i>Shift right logical</i>	rs1.H[0]>>rs2[3:0], rs1.H[1]>>rs2[3:0]
SRLI16 rd,rs1,imm4	<i>Shift right logical immediate</i>	rs1.H[0]>>imm4, rs1.H[1]>>imm4
SLL16 rd,rs1,rs2	<i>Shift left logical</i>	rs1.H[0]<<rs2[3:0], rs1.H[1]<<rs2[3:0]
SLLI16 rd,rs1,imm4	<i>Shift left logical immediate</i>	rs1.H[0]<<imm4, rs1.H[1]<<imm4

- Versões de 8 bits: SRA8, SRAI8, SRL8, SRLI8, SLL8, SLLI8; o valor imediato é de 3 bits.

Instruções de comparação

Mnemônica		Instrução	Resultado
CMPEQ16	rd, rs1, rs2	Comparação de igualdade	Verdade: 0xffff, Falso: 0x0000
SCMPLT16	rd, rs1, rs2	Menor que (com sinal)	
SCMPLE16	rd, rs1, rs2	Menor ou igual a (com sinal)	
UCMPLT16	rd, rs1, rs2	Menor que (sem sinal)	
UCMPLE16	rd, rs1, rs2	Menor ou igual a (sem sinal)	

- Versões de 8 bits: CMPEQ8, SCMPLT8, SCMPLE8, UCMPLT8, UCMPLE8.
O resultado tem 8 bits (0xff ou 0x00).

Instruções de multiplicação (16 bits)

Mnemônica	Operação	Resultado
SMUL16 rd,rs1,rs2	Multiplicação de 16 bits (com sinal)	$r[dL] = rs1.H[0] * rs2.H[0],$ $r[dU] = rs1.H[1] * rs2.H[1]$
SMULX16 rd,rs1,rs2	Multiplicação de 16 bits (com sinal) operação cruzada	$r[dL] = rs1.H[0] * rs2.H[1],$ $r[dU] = rs1.H[1] * rs2.H[0]$
UMUL16 rd,rs1,rs2	Multiplicação de 16 bits (sem sinal)	$r[dL] = rs1.H[0] * rs2.H[0],$ $r[dU] = rs1.H[1] * rs2.H[1]$
UMULX16 rd,rs1,rs2	Multiplicação de 16 bits (sem sinal) operação cruzada	$r[dL] = rs1.H[0] * rs2.H[1],$ $r[dU] = rs1.H[1] * rs2.H[0]$

- Especificação de pares de registros:
 - $r[dL]$: registo $2*d$, $r[dU]$: registo $(2*d+1)$

Instruções de multiplicação (8 bits)

Mnemônica	Operação	Resultado
SMUL8 rd,rs1,rs2	Multiplicação de 8 bits (com sinal)	$r[dL].H[0] = rs1.B[0] * rs2.B[0];$ $r[dL].H[1] = rs1.B[1] * rs2.B[1];$ $r[dU].H[0] = rs1.B[2] * rs2.B[2];$ $r[dU].H[1] = rs1.B[3] * rs2.B[3];$
SMULX8 rd,rs1,rs2	Multiplicação de 8 bits (com sinal) operação cruzada	$r[dL].H[0] = rs1.B[0] * rs2.B[1];$ $r[dL].H[1] = rs1.B[1] * rs2.B[0];$ $r[dU].H[0] = rs1.B[2] * rs2.B[3];$ $r[dU].H[1] = rs1.B[3] * rs2.B[2];$
UMUL8 rd,rs1,rs2	Multiplicação de 8 bits (sem sinal)	$r[dL].H[0] = rs1.B[0] * rs2.B[0];$ $r[dL].H[1] = rs1.B[1] * rs2.B[1];$ $r[dU].H[0] = rs1.B[2] * rs2.B[2];$ $r[dU].H[1] = rs1.B[3] * rs2.B[3];$
UMULX8 rd,rs1,rs2	Multiplicação de 8 bits (sem sinal) operação cruzada	$r[dL].H[0] = rs1.B[0] * rs2.B[1];$ $r[dL].H[1] = rs1.B[1] * rs2.B[0];$ $r[dU].H[0] = rs1.B[2] * rs2.B[3];$ $r[dU].H[1] = rs1.B[3] * rs2.B[2];$

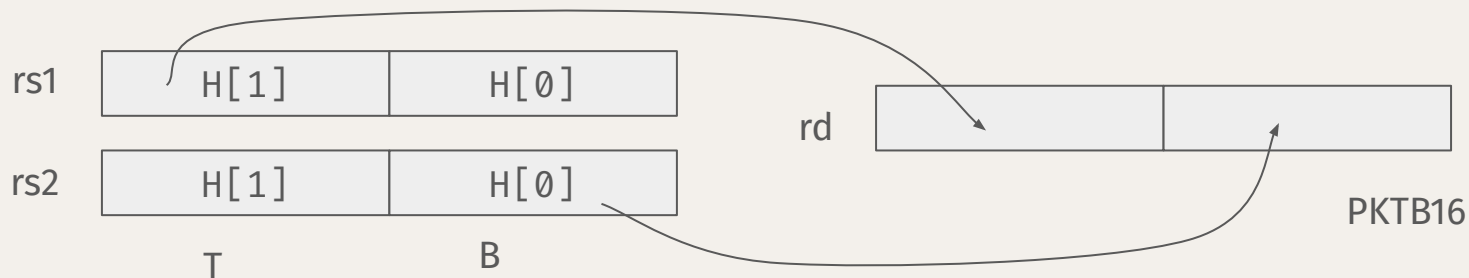
Instruções várias

Mnemônica	Operação	Resultado
SMIN16 rd,rs1,rs2	Menor de 2 números de 16 bits (com sinal)	$rd.H[x] = \min(rs1.H[x], rs2.H[x])$ para $x=0$ ou $x=1$
UMIN16 rd,rs1,rs2	Menor de 2 números de 16 bits (sem sinal)	$rd.H[x] = \min(rs1.H[x], rs2.H[x])$ para $x=0$ ou $x=1$
SMAX16 rd,rs1,rs2	Maior de 2 números de 16 bits (com sinal)	$rd.H[x] = \max(rs1.H[x], rs2.H[x])$ para $x=0$ ou $x=1$
UMAX16 rd,rs1,rs2	Maior de 2 números de 16 bits (sem sinal)	$rd.H[x] = \max(rs1.H[x], rs2.H[x])$ para $x=0$ ou $x=1$
SWAP16 rd,rs1	Trocar as duas meias palavras de uma palavra	$rd = \text{CONCAT}(rs1.H[0], rs1.H[1])$

- Versões de 8 bits: SMIN8, UMIN8, SMAX8, UMAX8, SWAP8 (troca bytes dentro de meias palavras).

Empacotamento de dados

Mnemônica	Operação	Resultado
PKBB16 rd,rs1,rs2	Juntar duas meias palavras de 2 registros (partes menos significativas)	$rd = \text{CONCAT}(rs1.H[0], rs2.H[0])$
PKTT16 rd,rs1,rs2	Juntar duas meias palavras de 2 registros (partes mais significativas)	$rd = \text{CONCAT}(rs1.H[1], rs2.H[1])$
PKBT16 rd,rs1,rs2	Juntar duas meias palavras de 2 registros	$rd = \text{CONCAT}(rs1.H[0], rs2.H[1])$
PKTB16 rd,rs1,rs2	Juntar duas meias palavras de 2 registros	$rd = \text{CONCAT}(rs1.H[1], rs2.H[0])$



Desempacotamento de dados

Mnemônica	Operação	Resultado
SUNPKD810 rd, rs1	Desempacotar bytes 1 e 0 (com extensão de sinal)	rd.H[1] = rs1.B[1] rd.H[0] = rs1.B[0]
SUNPKD820 rd, rs1	Desempacotar bytes 2 e 0 (com extensão de sinal)	rd.H[1] = rs1.B[2] rd.H[0] = rs1.B[0]
SUNPKD830 rd, rs1	Desempacotar bytes 3 e 0 (com extensão de sinal)	rd.H[1] = rs1.B[3] rd.H[0] = rs1.B[0]
SUNPKD831 rd, rs1	Desempacotar bytes 3 e 1 (com extensão de sinal)	rd.H[1] = rs1.B[3] rd.H[0] = rs1.B[1]
SUNPKD832 rd, rs1	Desempacotar bytes 3 e 2 (com extensão de sinal)	rd.H[1] = rs1.B[3] rd.H[0] = rs1.B[2]

- As versões equivalentes **com extensão de 0** são:
ZUNPKD810, ZUNPKD820, ZUNPKD830, ZUNPKD831, ZUNPKD832.

Exemplos

Exemplo 1: maiores valores de dois vetores

```
.data
V1: .byte 23,11,65,71,12,34,56,67
V2: .byte 40,26,61,71,11,71,65,56
V3: .space 8
.text
    la    s0, V1
    la    s1, V2
    la    s2, V3
    li    s3, 2  # porquê?
.L0:    beq    s3,zero, .L1
```

```
    lw    t0, 0(s0)
    lw    t1, 0(s1)
    umax8 t2, t0, t1
    sw    t2, 0(s2)
    addi   s0, s0, 4
    addi   s1, s1, 4
    addi   s2, s2, 4
    addi   s3, s3, -1
    b      .L0
.L1:    # ...
```

Resultado: 40,26,65,71,12,71,65,67

Exemplo 2: contar números ímpares (versão escalar)

.data

V1: .byte 15,18,20,2,5,6,5,1

.text

```
    la    s0, V1
    li    t0, 8
    mv     t5, zero # resultado
.L0:  lbu   t2, 0(s0)
      andi  t3, t2, 0x01
      add   t5, t5, t3
      addi  t0, t0, -1
      addi  s0, s0, 1
      bne   t0, zero, .L0
```

Número de instruções
executadas para um
vetor de N bytes:

$$3 + 6 \times N$$

Exemplo 2: contar números ímpares (versão SIMD)

```
.data
V1: .byte 15,18,20,2,5,6,5,1
```

```
.text
```

```
la      s0, V1
li      t0, 2
# máscara para 4 bytes
li      t1, 0x01010101
mv      t5, zero
```

```
.L0:   lw      t2, 0(s0)
        and     t3, t2, t1
        add8    t5, t5, t3
```

```
addi    t0, t0, -1
addi    s0, s0, 4
bne     t0, zero, .L0
```

```
# combinar parciais
mv      a0, zero
.L2:    andi    t6, t5, 0xFF
        add     a0, a0, t6
        srli    t5, t5, 8
        bne     t5, zero, .L2
# feito
```

Exemplo 2: Número de instruções executadas

Número de instruções executadas para um vetor de N bytes

- Versão escalar: $3 + 6 \times N$
- Versão SIMD: $4 + 6 \times N/4 + 1 + 4 \times 4 = 21 + 3/2 \times N = 21 + 1,5 \times N$

N	escalar	SIMD	escalar/SIMD
4	27	27	1
8	51	33	1,55
40	243	81	3
400	2403	621	3,87
4000	24003	6021	3,99

Para $N \rightarrow \infty$, desempenho de versão SIMD tende a ser $6/1,5 = 4\times$ melhor (menos instruções)

Representação digital de imagens



$F(x, y)$



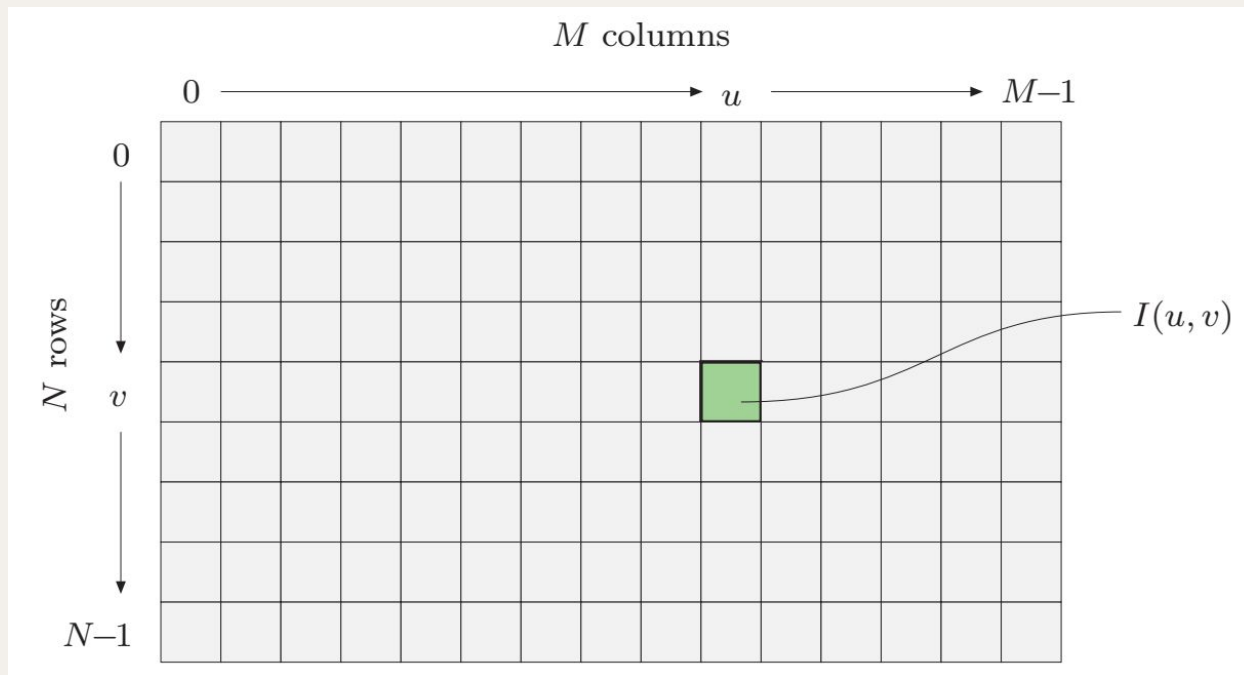
148	123	52	107	123	162	172	123	64	89	...
147	130	92	95	98	130	171	155	169	163	...
141	118	121	148	117	107	144	137	136	134	...
82	106	93	172	149	131	138	114	113	129	...
57	101	72	54	109	111	104	135	106	125	...
138	135	114	82	121	110	34	76	101	111	...
138	102	128	159	168	147	116	129	124	117	...
113	89	89	109	106	126	114	150	164	145	...
120	121	123	87	85	70	119	64	79	127	...
145	141	143	134	111	124	117	113	64	112	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

$I(u, v)$



Wilhelm Burger & Mark J. Burge, "Principles of Digital Image Processing: Fundamental Techniques", 2009, Springer.

Coordenadas



Wilhelm Burger & Mark J. Burge, "Principles of Digital Image Processing: Fundamental Techniques", 2009, Springer.

Valores de pixels

Grayscale (Intensity Images):

<i>Chan.</i>	<i>Bits/Pix.</i>	<i>Range</i>	<i>Use</i>
1	1	0...1	Binary image: document, illustration, fax
1	8	0...255	Universal: photo, scan, print
1	12	0...4095	High quality: photo, scan, print
1	14	0...16383	Professional: photo, scan, print
1	16	0...65535	Highest quality: medicine, astronomy

Wilhelm Burger & Mark J. Burge, "Principles of Digital Image Processing: Fundamental Techniques", 2009, Springer.

Exemplo 3: Binarizar uma imagem (tons de cinzento)

- Imagem com 640 colunas e 480 linhas (resolução)
- Tons de cinzento: 1 byte / pixel
- N. de pontos: 307200
- Pontos com $I(u,v) > 127 \rightarrow 255$
- Pontos com $I(u,v) < 128 \rightarrow 0$

Exemplo 3: versão SIMD

```
.data
imagem:    .byte 148, 123, ...
nbytes:    .word 307200
```

```
.text
    la      s0, imagem
    lw      t0, nbytes
    srli    t0, t0, 2
    li      t1, 0x7F7F7F7F
```

```
.L0:    lw      t2, 0(s0)
        ucmple8 t3, t2, t1
        xori    t3, t3, -1
        sw      t3, 0(s0)
        addi    s0, s0, 4
        addi    t0, t0, -1
        bne     t0, zero, .L0
        # feito ...
```